

1 OCR Rockfest is a yearly music festival. Visitors buy tickets to attend and watch the music acts.

A computer system is created to sell tickets and store information about visitors and the music acts.

Tickets to the festival are £60 each. Visitors can buy multiple tickets. For example, a visitor buying 4 tickets would pay £240.

i. Write a function, `calculatePrice()` that:

- takes the number of tickets to be purchased as a parameter
- calculates and returns the total price to pay.

You must use either:

- OCR Exam Reference Language, or
- a high-level programming language that you have studied.

[4]

ii. Write an algorithm that:

- uses the function `calculatePrice()` to find the price for a visitor buying 3 tickets
- stores the result in the variable `totalPrice`

You must use either:

- OCR Exam Reference Language, or
-

a high-level programming language that you have studied.

- 2 An algorithm stores the position of a character on a straight line as an integer. A user can move the character left or right.

The following algorithm:

- generates one random number between 1 and 512 (inclusive) to store as the position
- prompts the user to input a direction to move (left or right)
- takes a direction as input until a valid direction is input.

```
p = random(1, 512)
```

```
print("The position is ", p)
```

```
a = ""
```

```
while a != "left" and a != "right"
```

```
    a = input("Enter direction, left or right")
```

```
endwhile
```

If the character moves left, 5 is subtracted from the position.

If the character moves right, 5 is added to the position.

The position of the character can only be between 1 and 512 inclusive.

The function `moveCharacter()` takes the direction (left or right) and current position as parameters

- changes position based on direction
- sets position to 1 if the new position is less than 1
- sets position to 512 if the new position is greater than 512
- returns the new position.

Complete the function `moveCharacter()`

```
function moveCharacter(direction, position)
```

[illegible]

[6]

3 OCR Drones flies goods around the country using drones.

A text file is used to store pilot's codes and date of birth.

The text file is named `pilots.txt`. The contents of some of this file is shown:

JP670,17/05/1986

CC200,31/06/1980

SM720,01/04/1976

GC730,06/06/1978

Design an algorithm to:

- Ask the user to input a pilot code and a date of birth
- Write these inputs to the `pilots.txt` text file

[4]

END OF QUESTION PAPER

Question			Answer/Indicative content	Marks	Guidance
1		i	- Function definition for <code>calculatePrice()</code> ... with parameter <u>that is not overwritten</u> - Calculate price based on multiplying parameter (if present) by 60<u>always return calculated price</u>	4	Section B so must be high level code. Do not accept x for multiplication. Do not accept spaces in function, variable or parameter names if clearly obvious. <u>Example code</u> <pre>function calculatePrice(num) price = num * 60 return price endfunction</pre> All code must be within function definition (if present). BP3 – if no parameter present, allow multiplying by sensible value (e.g. input value). FT overwriting parameter. BP4 must be return. Do not credit output/print. Value returned can be with or without £ sign and other text. FT for BP4 if sensible attempt made at calculation. Accept multiple parameters passed in. Ignore superfluous code Accept returning by assigning to function name (e.g. <code>calculatePrice = price</code>), some languages such as VB do this. <u>Examiner's Comments</u> These questions proved to be excellent discriminators, with only candidates who achieved very highly overall able to do well. Although teaching of programming by centres has improved students' general overall responses, it is clear that defining functions and then utilising them is an area that still requires focus. For part (c) (ii), candidates were asked to define a function. Many candidates were able to begin to define a function. However, a common misconception was either the lack of a parameter in the definition, or overwriting/ignoring the parameter and asking for separate input within the function. Another very common misconception was to print or otherwise output the calculated price instead of returning it as required. For part c (iii), candidates were asked to

Question			Answer/Indicative content	Marks	Guidance
					then use this function to calculate the price of 3 tickets and store the answer in specified variable. Very few candidates did this successfully. Instead many candidates attempted to redefine the function again or to work out the price of the tickets in some other way.  Misconception When a function is defined, all data passed to the function should be done via parameters. It is not appropriate to ask for input inside the function, especially when this overwrites or replaces an existing parameter. Functions also return a value instead of printing anything, as clearly requested in the question. Exemplar 2 <pre>def calculatePrice(the tickets): tickets = int(input("Enter a number of tickets:")) price = tickets * 60 print("The total price is:", price) print("The total price is:", price)</pre> In this example, the candidate has successfully defined a function with a single parameter, tickets. This first line would have achieved mark points 1 and 2. However, the tickets' parameter is then overwritten with an unnecessary input statement. The price is then correctly calculated but is printed instead of returned. As a result, this example receives 2 out of 4 marks.

Question			Answer/Indicative content	Marks	Guidance
		ii	• CalculatePrice(3) called... • ... return value assigned to totalPrice	2	<u>Example code</u> <pre>totalPrice = calculatePrice(3)</pre> Do not accept == for assignment Allow FT to BP2 only if sensible attempt at BP1 (e.g. missing/wrong parameter) Allow multiple values passed in as long as 3 is one of the values. Do not accept definitions of a function – must be calling the existing function. Penalise spaces in variable / function name only if clearly obvious. Ignore superfluous code. Ignore case. <u>Examiner's Comments</u> These questions proved to be excellent discriminators, with only candidates who achieved very highly overall able to do well. Although teaching of programming by centres has improved students' general overall responses, it is clear that defining functions and then utilising them is an area that still requires focus. For part (c) (ii), candidates were asked to define a function. Many candidates were able to begin to define a function. However, a common misconception was either the lack of a parameter in the definition, or overwriting/ignoring the parameter and asking for separate input within the function. Another very common misconception was to print or otherwise output the calculated price instead of returning it as required. For part c (iii), candidates were asked to then use this function to calculate the price of 3 tickets and store the answer in specified variable. Very few candidates did this successfully. Instead many candidates attempted to redefine the function again or to work out the price of the tickets in some other way.  Misconception

Question			Answer/Indicative content	Marks	Guidance
					When a function is defined, all data passed to the function should be done via parameters. It is not appropriate to ask for input inside the function, especially when this overwrites or replaces an existing parameter. Functions also return a value instead of printing anything, as clearly requested in the question. Exemplar 2 <pre>def calculatePrice(the tickets): tickets = int(input("Enter the number of tickets:")) price = tickets * 60 print("The total price is:", price) print("The total price is:£", price)</pre> In this example, the candidate has successfully defined a function with a single parameter, tickets. This first line would have achieved mark points 1 and 2. However, the tickets' parameter is then overwritten with an unnecessary input statement. The price is then correctly calculated but is printed instead of returned. As a result, this example receives 2 out of 4 marks.
			Total	6	

Question	Answer/Indicative content	Marks	Guidance
2	1 mark each to max 6 • Appropriate use of both parameters and no additional inputs / incorrect overwrites that affect outcome of algorithm • Attempt at selection... • ...correctly checking if direction is "left" and subtracting 5 from position (or equivalent) • ...correctly checking if direction is "right" and adding 5 to position (or equivalent) • Ensuring position (or equivalent) is between 1 and 512 inclusive • Returning the updated position <u>Example</u> <pre> if direction == "left" then position = position - 5 elseif direction == "right" then position = position + 5 endif if position < 1 then position = 1 elseif position > 512 then position = 512 endif return position </pre>	6 (AO3)	Allow else for BP3/4 (validated in question 8a) Allow <=, >= and equivalents (e.g. <= 0) for BP5. Do not award BP5 if before BP3 and 4 (otherwise will alter position value) BP6 only to be given if attempt made at calculating new position. Calculation can be partial/incorrect. Ignore repeat of function header / end. Accept flowchart / structured English but must not just repeat the question. If response uses loop to incorrectly change position multiple times, do not award BP1 (incorrect overwrite) For minor syntax errors (e.g. missing quotation marks or == for assignment, spaces in variable names) penalise once then FT. <u>Examiner's Comments</u> This question was an excellent discriminator in terms of candidate achievement. In particular, correct use of the given parameters (direction and position) was only seen from the most successful candidates. Many candidates instead started their response by asking for input from the user and therefore overwriting the parameters, losing crucial marks in the process. As on previous papers, this question provided one mark for any attempt made at a section of the requirement, in this case selection. Candidates who attempted to use an if or select case statement, even incorrectly or in the wrong context were able to achieve at least this mark; centres should therefore continue to encourage all candidates to attempt each and every question and not leave responses blank. It was challenging to achieve full marks, but many candidates did because of their excellent levels of practical programming

Question			Answer/Indicative content	Marks	Guidance
					experience in school. Exemplar 2 <pre>function moveCharacter(direction, position) if direction == "left": position -= 5 if direction == "right": position += 5 if position < 1: position = 1 if position > 512: position = 512 return position</pre> This candidate achieved full marks. The given parameters (direction and position) are both used and not overwritten by inputs before the position is modified depending on the direction given. The position is then validated to make sure that it is between 1 and 512 before being returned. Note the use of position +=5 to add 5 to the position. This is an entirely acceptable alternative to position = position + 5
			Total	6	

Question			Answer/Indicative content	Marks	Guidance
3			One mark per bullet point • Ask the user for two inputs and store/use these • Open pilots.txt (for write/append) • Write both inputs to opened text file • Close text file	4 (AO3 2b)	Award BP4 for implicit closing of file (e.g. using with... in Python) Example answer : <pre>dronepilotData = open("pilots.txt") pilotCode = input("enter code") dob = input("enter date of birth") dronepilotData.writeLine(pilotC ode, dob) dronepilotData.close()</pre> Allow data to be written either by simply writing both values or by concatenating into one string with a separating comma.
			Total	4	